RetailForge

Your AI Personal Shopper — that actually shops.

A production-grade **multi-agent retail intelligence** system: a department-store storefront with a concierge that searches, recommends, builds discounted kits, places orders and issues refunds — with safe, gated access to live data.

Live demo: retailforge-frontend-awghszkm2a-uc.a.run.app

Google ADK · Gemini 2.5 Flash · MCP Toolbox · MongoDB Atlas Vector Search · Cloud Run

Devpost Hackathon Submission

⋆ The problem

Retail chatbots answer. They don't act.

- Shoppers get text replies, then still hunt, compare and check out alone.
- "Recommendations" rarely know real *stock*, *price* or *promotions*.
- Bundling and discounts are manual work the shopper does in their head.

Letting an LLM touch the database is risky.

- Direct DB access = an agent one hallucination away from a destructive write.
- Hard to audit *what* an agent actually did to your data.
- No clean contract between "the model" and "the system of record".

The gap

We need a concierge that **takes real, correct actions** on live retail data — *and* a safety boundary that makes those actions auditable and impossible to abuse.

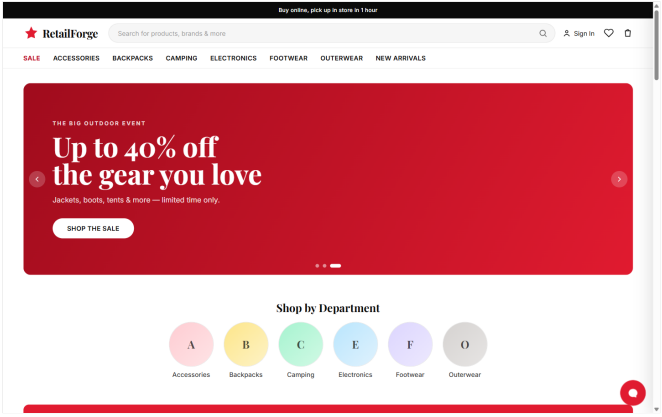
⋆ What RetailForge does

A shopper chats with one **Personal Shopper**. Behind it, a team of specialist agents do the work — end to end.

- **Find** — natural-language semantic search over the catalog (vector embeddings).
- **Recommend** — "also bought", trending, and history-aware picks.
- **Support** — order status, cancellations, refunds.
- **Build a kit** — assemble a multi-category bundle for an activity…
- **…and discount it** — auto-pick the best valid promotion.
- **Check out** — place the order & decrement inventory for real.

"*Build me a weekend camping kit under $300 with a discount*" ⇒ the concierge routes to the Billing agent, searches across categories, checks stock, applies a promo, and returns a ready-to-buy bundle.

★ The storefront — live on Cloud Run

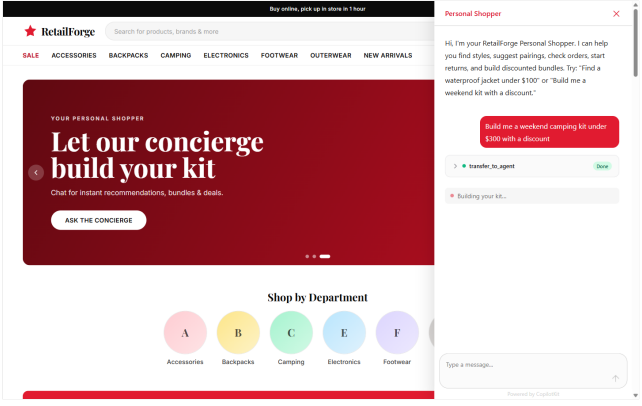


A polished **department-store** experience built in Next.js 15 + Tailwind:

- Hero carousel, department tiles, product rails.
- Rich product pages: reviews, stock by aisle, size/color.
- Wishlist, bag, and an order history page.

The red star wordmark, serif display type and bold sale banners give it an unmistakable retail-floor feel.

★ The concierge in action — generative UI

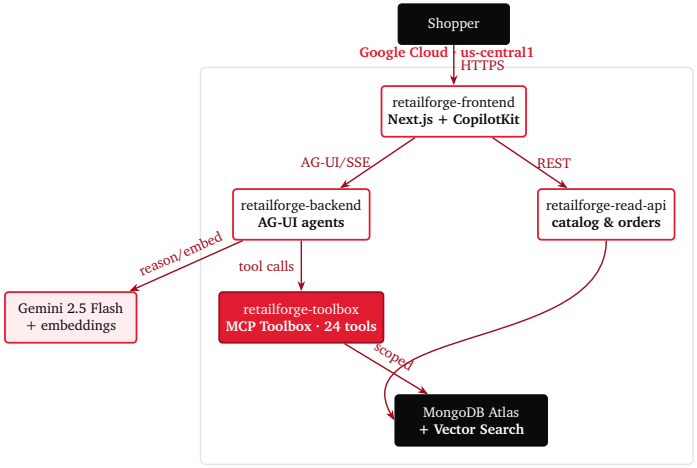


Powered by **CopilotKit** over the **AG-UI** protocol (server-sent events):

- The root agent visibly `transfer_to_agent` to a specialist. . .
- . . . which streams **generative-UI cards**: product grids, kit builder, order confirmations, discount results.
- Every card has live "Add to Bag" actions wired to the same state as the store.

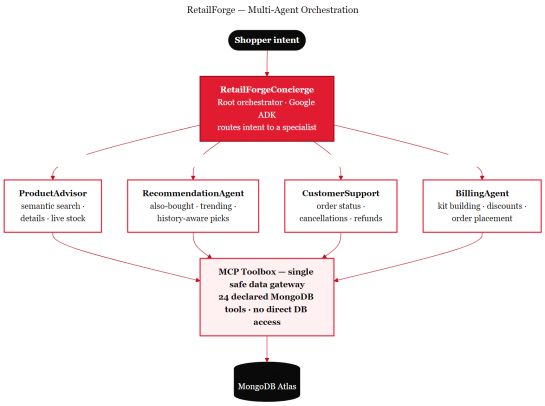
Captured live — the agent hand-off is real, not a mock.

⋆ System architecture — four Cloud Run services



Each service is independently deployable; secrets are injected from Secret Manager, images from Artifact Registry.

⋆ Multi-agent orchestration



A **root concierge** (Google ADK) classifies intent and hands off to a specialist:

- **ProductAdvisor** — search, details, live stock.
- **Recommendation** — co-purchase, trending, personalization.
- **CustomerSupport** — status, cancel, refund.
- **Billing** — kit building, discounts, order placement.

Strategy lives in the LLM; deterministic math (pricing, discounts, inventory) stays in native code — not left to the model.

⋆ The key idea — MCP Toolbox as a safety boundary

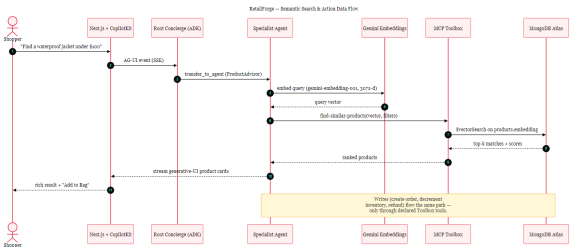
Agents never open a database connection.

- Every data operation is a **declared tool** in `tools.yaml`.
- 24 tools, grouped into 4 toolsets — one per specialist agent.
- An agent can only call what's declared: no ad-hoc, no "DROP".

Why it matters

- **Auditable** — every write is a named, logged tool call.
- **Least privilege** — read tools and write tools are explicit.
- **Clean contract** — the model and the system of record are decoupled.

⋆ Semantic search & the action data-flow



- Products embedded at seed time with `gemini-embedding-001` (**3072-d**, cosine).
- Queries embedded at run time; **Atlas $vectorSearch** returns top matches with score + metadata filters (category, brand, price).
- **Writes** (create order, decrement inventory, refund) travel the *same* gated Toolbox path.

⋆ Data model — one Atlas database

Collection	Holds
products	catalog + **embedding** vector (Vector Search index)
inventory	qty on hand, aisle (decremented on order)
orders	line items, subtotal, discount, status, refunds
reviews	ratings & text, aggregated per product
promotions	codes: percentage / fixed, min subtotal, min items
users	shopper profiles
carts	abandoned-cart items

Vertical-agnostic: swap `data/sample/*.json` to retarget the whole store to any retail category.

⋆ Cloud-native deployment

Four Cloud Run services

- **frontend** — Next.js storefront.
- **backend** — FastAPI AG-UI agent server.
- **read-api** — FastAPI catalog/orders.
- **toolbox** — distroless MCP Toolbox binary.

Managed building blocks

- Secret Manager — Mongo URI & Gemini key.
- Artifact Registry — SHA-tagged images.

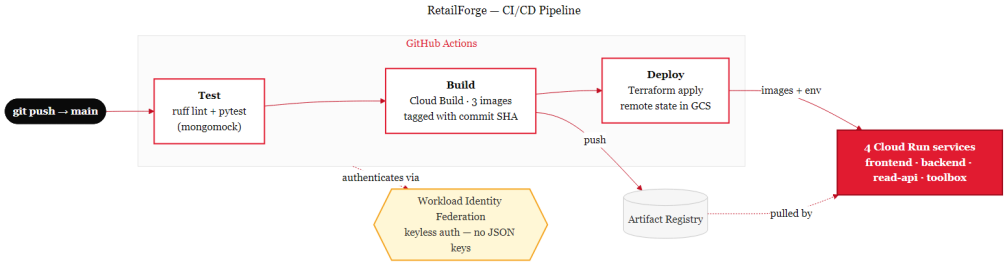
Infrastructure as code

- **Terraform** provisions all services, secrets & IAM.
- Remote state in GCS — repeatable applies.
- One `terraform apply` stands up the whole stack.

Live URLs

frontend / backend / read-api / toolbox
`retailforge-*-awghszkm2a-uc.a.run.app`

⋆ CI/CD — push to production, keyless

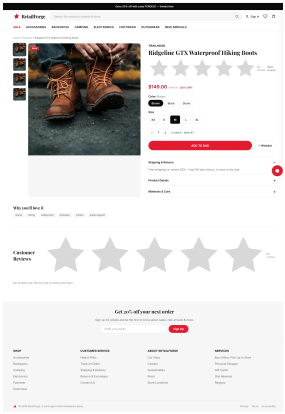


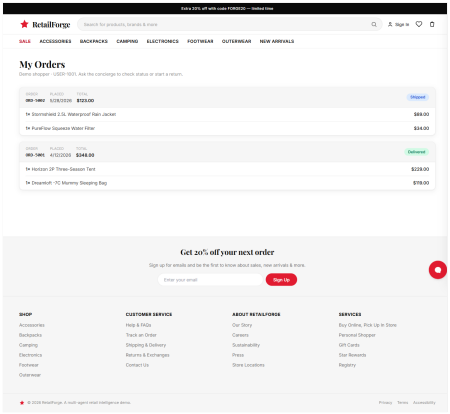
GitHub Actions: test (ruff + pytest on mongomock) → Cloud Build (3 images, commit-SHA tags) → Terraform apply. Auth via **Workload Identity Federation** — *no service-account JSON keys in the repo*.

⋆ Tech stack

Layer	Technology
Frontend	Next.js 15, React 19, TypeScript, Tailwind, CopilotKit, AG-UI
Agents	Google ADK, ag-ui-adk, Gemini 2.5 Flash
Backend	FastAPI, Uvicorn, Pydantic
Data	MongoDB Atlas + Vector Search, gemini-embedding-001 (3072-d)
Tooling	MCP Toolbox (genai-toolbox), toolbox-core
Infra	Cloud Run, Secret Manager, Artifact Registry, Cloud Build
IaC & CI/CD	Terraform, GitHub Actions, Workload Identity Federation
Quality	pytest, mongomock, ruff

⋆ More of the experience





Product detail with reviews & live stock · order history with status badges · fully responsive mobile.

⋆ Challenges & what we learned

Challenges

- Keeping agents from touching data directly — solved with the Toolbox boundary.
- Tuning vector search relevance (task-type, filters, top-k).
- Running a **distroless** Toolbox on Cloud Run (no shell, direct binary, port 8080).
- Boot ordering: make Toolbox public *before* the backend loads its tools.

Learnings

- Let the **LLM strategize**; keep money math in deterministic code.
- A declared-tool layer makes agent actions *auditable by construction*.
- Generative UI beats plain chat — actions live where the answer is.
- Keyless WIF + Terraform = clean, repeatable, secret-free deploys.

What’s next

Tighten IAM (auth’d service-to-service) + VPC egress for static Atlas IPs.

Personalization from real session & purchase history.

Payment + fulfillment integration; A/B test agent strategies.

Observability: trace every agent hand-off and tool call.

Try it live: retailforge-frontend-awghszkm2a-uc.a.run.app

Thank you